

Make Agents Observable

Tracing Decisions, Tool Calls and Recovery

Emerson Braun



Emerson Braun

Software Architect /
Regional Director
Plan A Technologies

15+ years in software engineering
Creator of AgentsKit

AGENT RESPONSE

**“Done.
The issue is resolved.”**

REALITY

Users still can't check out.

A confident answer is not evidence of success.



The same answer. Three different failures.

“Done.”



A diagram with the word "Done." at the top. Three lines branch out from below it: a green line to the left, a yellow line straight down, and a red line to the right. Each line points to a different failure category.

Wrong context

Used an
outdated policy.

Failed action

The refund never
went through.

Unverified outcome

Closed the case.
The customer is still
waiting.

A confident answer is still a claim.

THE AGENT SAYS

“The task is complete.”

EVIDENCE NEEDED

Context used: instructions and data

Route selected: action chosen

Outcome verified: real-world change

Keep the evidence behind the claim.

Observe the execution. Not just the answer.

Context → **Decision** → **Action** → **Outcome**

What information
did it use?

Why this
action?

What actually
happened?

Did it solve
the problem?

**The action was correct.
The policy was outdated.**

“Refund my purchase from 21 days ago.”

Policy used

Refund window: 30 days

Refund approved.

Current policy

Refund window: 14 days

Can you prove which version the agent used?

Technical success $\neq$ task success.

TASK EFFECT

Outcome verified

The intended state changed.

VALIDITY

Data usable

The response can be parsed and used.

RESPONSE

**Integration
replied**

The system received a response.

THE USER BENEFITS FROM THE THIRD CLAIM.
Technical signals explain the result. Task evidence proves it.

A failure is a state — not a conclusion.

1**FAILURE**

Preserve the error and execution state.

2**CLASSIFY**

Failed, timed out, or invalid result?

3**VERIFY STATE**

Check what actually changed.

4**CHOOSE RECOVERY**

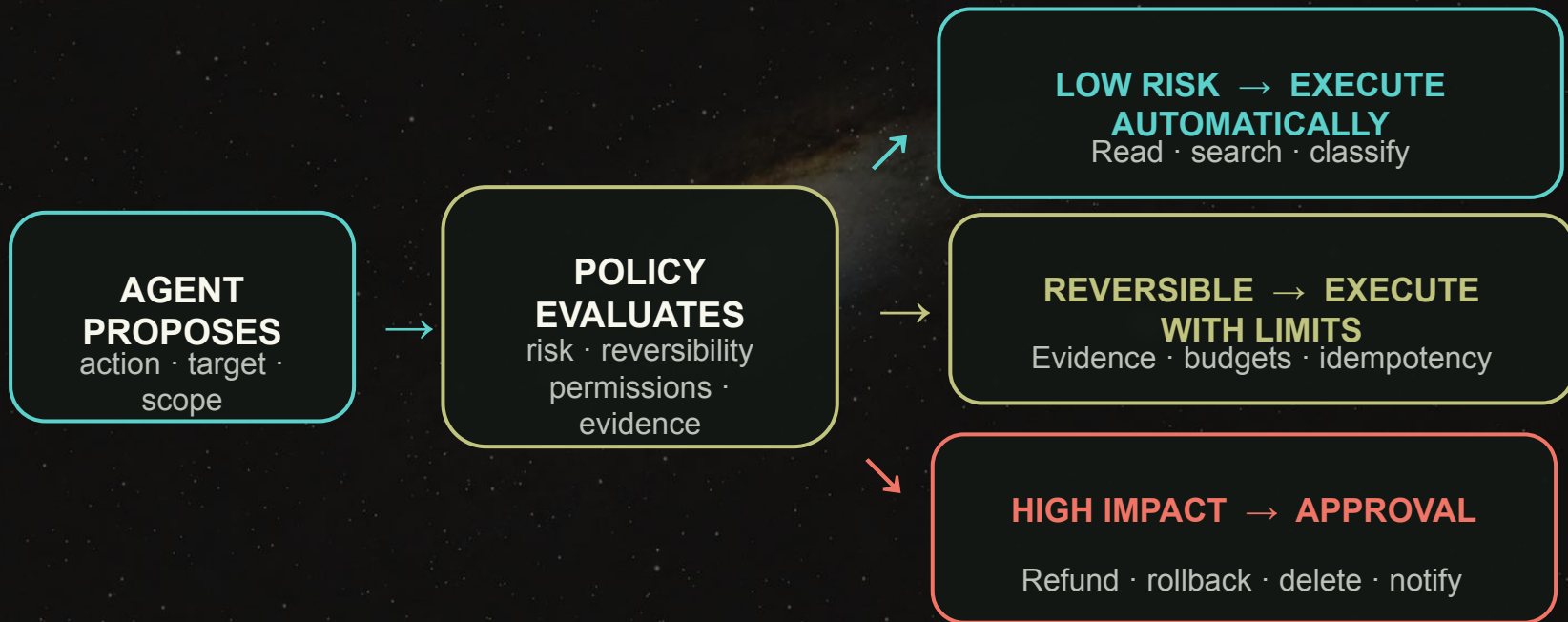
Retry, reroute, compensate, escalate.

5**PROVE OUTCOME**

Recovery ends with new evidence.

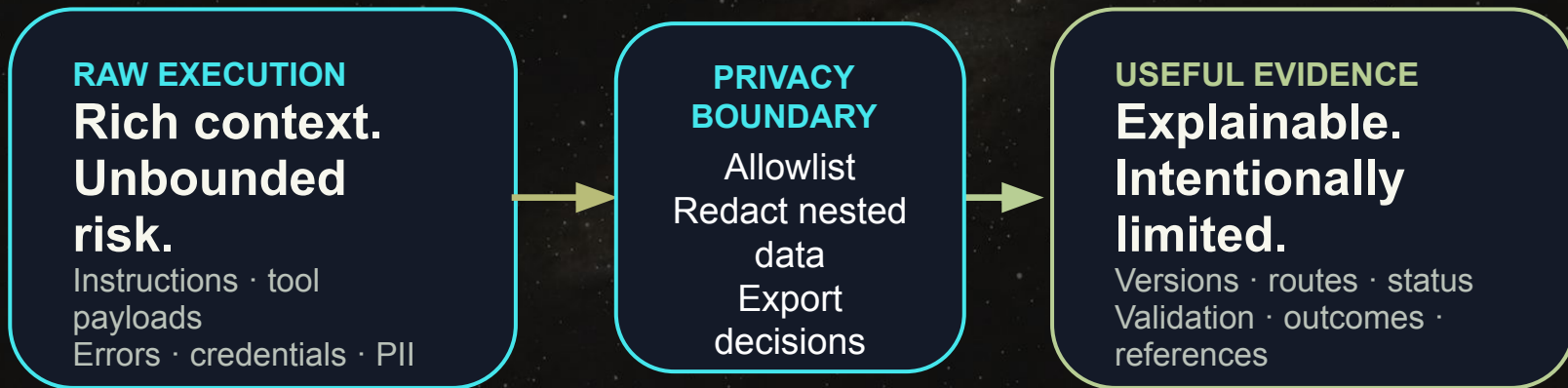
Do not automate retries. Automate recovery decisions.

Autonomy needs a control boundary.



Telemetry is evidence — and a liability.

Observability should preserve evidence, not copy the entire execution.



Collect enough to explain the system — not enough to recreate the user.

One model. Different investigations.

The domain changes. The evidence model does not.

SUPPORT

Did the refund
actually
happen?

AUTOMATION

Did the
deployment
recover?

CODE

Did validation
pass
without
regressions?

CONTEXT → DECISION → ACTION → VERIFIED OUTCOME

Ask the same operational questions in every domain.

The model is only part of the bill.

MODEL CALL
Tokens · model latency

TOOL CALLS

APIs · databases ·
search · compute

RETRIES & LOOPS

Every repeated step multiplies cost

WAITING & RECOVERY

Approvals · timeouts · fallbacks · diagnosis

Measure cost per successful outcome — not cost per model call.

Instrument your next workflow.

Ten checks before you call an agent observable.

01 Version instructions and context

02 Record structured route decisions

03 Trace every external boundary

04 Separate request, attempt and effect

05 Validate outputs semantically

06 Model recovery as explicit states

07 Bind approvals to exact actions

08 Redact before telemetry export

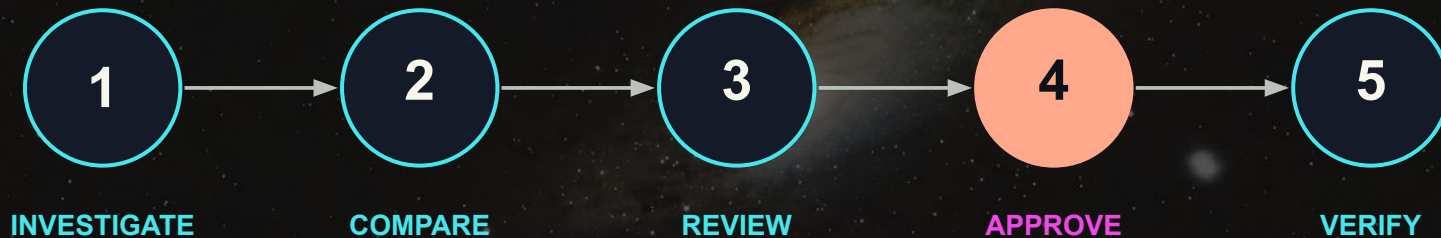
09 Measure verified outcomes

10 Link metrics back to traces

If you cannot reconstruct the execution, you cannot operate it.

Now, follow the evidence.

One incident. Two execution paths. One shared trace model.

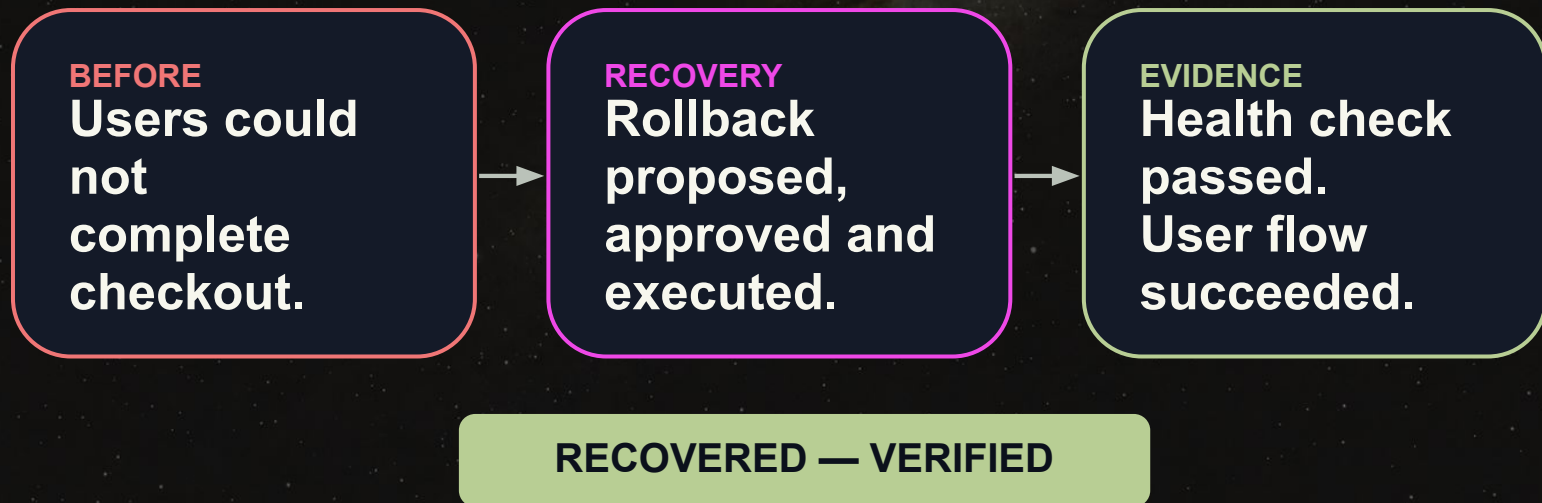


SCRIPTED ADAPTER + MOCK TOOLS · NO LIVE LLM · NO PRODUCTION CHANGES

The rollback requires a real approval click.

What proves it worked?

Return from the demo with evidence, not confidence.



The green badge is not the proof. The verified outcome is.

Don't ask only what the agent said.

Ask what happened — and what proves it worked.

**CONTEXT → DECISION → ACTION
→ OUTCOME → EVIDENCE**



LINKEDIN

linkedin.com/in/emerson-braun



LIVE DEMO

AgentsKit
observability demo

Emerson Braun · AgentsKit